UNIT IV
Multithreading & Java Database Connectivity
IT405PC JAVA PROGRAMMING

Shafakhatullah Khan Mohammed
Department of Information Technology

ACE Engineering College

Multithreading | JDBC Connectivity

Table of Contents

PART A

Multithreading in Java

Process · Thread · Life Cycle · Creating Threads · Priority · Synchronization

What is a Process?

Definition

A **Process** is an independent program in execution that has its own memory space, resources, and execution environment allocated by the Operating System.

Key Characteristics:

- Has its **own separate memory** space
- Has its own file handles & registers
- Runs **independently** from other processes
- Communicates via **IPC**
- **High overhead** to create and manage

Key Point

Real World Analogy:

Think of a **Process** like a **factory** — it has its own building, machinery, and workers. Each factory operates completely independently.

What is a Thread?

Definition

A **Thread** is the smallest unit of execution within a process. Multiple threads can exist within a single process, sharing the same memory space and resources.

Key Characteristics:

- **Shares memory** with other threads
- Shares resources of parent process
- **Lightweight** – less overhead
- Can directly communicate with other threads
- **Faster** to create and switch

Key Point

Real World Analogy:

Think of a **Thread** like a **worker inside a factory** — multiple workers share the same building and tools but perform different tasks simultaneously.

Process-Based vs Thread-Based Multitasking

Definition

Multitasking is the ability to execute multiple tasks simultaneously. Java supports both **Process-based** and **Thread-based** multitasking.

Feature	Process-Based	Thread-Based
Memory	Separate memory per process	Shared memory among threads
Communication	Complex (IPC, pipes, sockets)	Simple (shared variables)
Overhead	Heavy (more resources)	Light (fewer resources)
Switching	Expensive context switch	Fast context switch
Isolation	Fully isolated	Not fully isolated
Example	MS Word & Chrome running together	Spell-check while typing
Java Support	`Runtime.exec()`	`Thread` / `Runnable`

Definition

The **Thread Life Cycle** describes the various **states** a thread passes through from its creation to its termination.

State	Description	How to Enter
New	Thread object created, not started	`new Thread()`
Runnable	Ready to run, waiting for CPU	`t.start()`
Running	Currently being executed	JVM scheduler assigns CPU
Blocked	Temporarily inactive	`sleep()`, `wait()`, `join()`
Terminated	Execution completed	`run()` finishes

Creating Threads – Method 1: Extending Thread Class

Definition

Java provides **two ways** to create threads:
1. By **extending** the `Thread` class **2.** By **implementing** the `Runnable` interface

Syntax – Extending Thread Class:

```
1 class MyThread extends Thread {
2 public void run () {
3 // Code to execute in this thread
4 }
5 }
6
7 // Creating and Starting the Thread :
8 MyThread obj = new MyThread ();
9 obj . start (); // Calls run () internally -- DO NOT call run () directly
```

Creating Threads – Method 2: Implementing Runnable

Syntax – Implementing Runnable Interface:

```
1 class MyRunnable implements Runnable {
2 public void run () {
3 // Code to execute in this thread
4 }
5 }
6 // Creating and Starting : MyRunnable
7 r = new MyRunnable () ;
8
9 Thread t = new Thread (r); // Pass Runnable to Thread constructor
10 t. start () ;
```

Thread vs Runnable – Comparison:

Aspect	extends Thread	implements Runnable
Inheritance	Cannot extend any other class	Can extend another class ✔
Flexibility	Less flexible	More flexible ✔
OOP Design	Violates single inheritance	Follows good OOP design ✔
Recommended?	Less preferred	✔ Preferred approach

Thread Example Program

```
public class ThreadDemo {
    public static void main(String[] args) {
        // Using Thread class
        MyThread t1, t2; t1 = new MyThread("Thread-A");
        t2 = new MyThread("Thread-B");
        // Using Runnable interface
        Thread t3 = new Thread(new MyRunnable("Thread-C"));
        // Start threads
        t1.start(); t2.start(); t3.start();
    }
}
class MyThread extends Thread {
    String threadName;
    MyThread(String name) { this.threadName = name; }
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(threadName + " - Count: " + i);
            try { Thread.sleep(500); } catch (InterruptedException e) { System.out.println(e); }
        } System.out.println(threadName + " Finished!");
    }
}
class MyRunnable implements Runnable {
    String threadName;
    MyRunnable(String name) { this.threadName = name; }
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(threadName + " - Count: " + i);
            try { Thread.sleep(300); } catch (InterruptedException e) { System.out.println(e); }
        } System.out.println(threadName + " Finished!");
    }
}
```

Thread Example – Output & Explanation

Sample Output:

```
Thread-A - Count: 1
Thread-C - Count: 1
Thread-B - Count: 1
Thread-C - Count: 2
Thread-A - Count: 2
Thread-B - Count: 2
Thread-C - Count: 3      Thread-C Finished!
Thread-A - Count: 3      Thread-A Finished!
Thread-B - Count: 3      Thread-B Finished!
```

Code	Explanation
`extends Thread`	Class inherits Thread to become a thread
`implements Runnable`	Class implements Runnable interface
`run()`	Contains the code the thread executes
`Thread.sleep(500)`	Pauses current thread for 500 ms
`InterruptedException`	Exception thrown if thread is interrupted
`t1.start()`	Registers thread with JVM; calls run() internally
`new Thread(Runnable Object)`	Wraps Runnable inside a thread object

Thread Priorities

Definition

Thread Priority is a value that tells the thread scheduler which thread should be given **preference** when multiple threads are ready to run. Higher priority threads get more CPU time.

```
Priority Constants:
-------------------
1 Thread . MIN_PRIORITY = 1
2 Thread . NORM_PRIORITY = 5 // Default
3 Thread . MAX_PRIORITY = 10

Syntax:
-------
1 Thread t = new Thread ();
2 // Set priority
3 t. setPriority ( Thread . MAX_PRIORITY );
4 t. setPriority (7) ;
5 // Get priority
6 int p = t. getPriority ();
```

Priority Methods:

Method	Description
`setPriority(int p)`	Sets priority (1–10)
`getPriority()`	Returns current priority

Important Note

Thread priority is a **hint** to the scheduler – it is **NOT guaranteed**. Actual behavior depends on the OS and JVM implementation.

Thread Priority – Example Program

```
class PriorityThread extends Thread {
    String name;
    PriorityThread(String name) { this.name = name; }
    public void run() {
        System.out.println(name + " | Priority: " + getPriority() + " | Running...");
        for (int i = 1; i <= 3; i++) { System.out.println(name + " -> Step " + i); }
    }
}
public class ThreadPriorityDemo {
    public static void main(String[] args) {
        PriorityThread t1 = new PriorityThread("LOW Thread");
        PriorityThread t2 = new PriorityThread("NORMAL Thread");
        PriorityThread t3 = new PriorityThread("HIGH Thread");
        // Setting priorities
        t1.setPriority(Thread.MIN_PRIORITY);   // 1
        t2.setPriority(Thread.NORM_PRIORITY);  // 5
        t3.setPriority(Thread.MAX_PRIORITY);   // 10
        // Main thread priority
        System.out.println("Main Priority: " + Thread.currentThread().getPriority());
        // Start threads
        t1.start(); t2.start(); t3.start();
    }
}
```

Synchronizing Threads

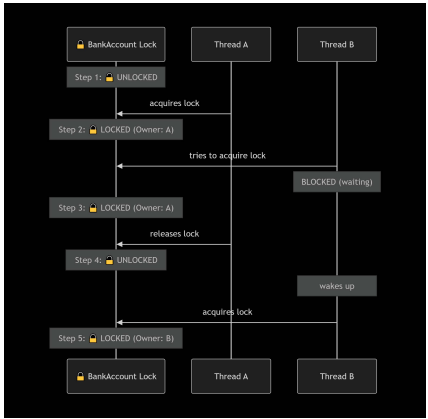
Definition

Synchronization is the mechanism that ensures that **only one thread at a time** can access a shared resource, preventing **data inconsistency** and **race conditions**.

```
Problem Without Synchronization:
--------------------------------
1 Thread A reads balance = 1000
2 Thread B reads balance = 1000
3 Thread A withdraws 500 -> 500
4 Thread B withdraws 500 -> 500
5 // Final : 500 ( Should be 0!)

Synchronized Method Syntax:
---------------------------
1 synchronized returnType method () {
2 // One thread at a time
3 }

Synchronized Block Syntax:
--------------------------
1 synchronized (object) {
2 // critical section (shared resource)
3 }
```



Synchronization – Example Program

```
class BankAccount {
    private int balance = 1000;
    public synchronized void withdraw(int amount) {
        System.out.println(Thread.currentThread().getName() + " trying to withdraw : " + amount);
        if (balance >= amount) {
            try { Thread.sleep(100); } catch (InterruptedException e) {} balance -= amount;
            System.out.println(Thread.currentThread().getName() + " Success ! Balance : " + balance);
        } else {
            System.out.println(Thread.currentThread().getName() + " Insufficient Balance ! Balance : " + balance);
        }
    }
}
class Customer extends Thread {
    BankAccount account; int amount;
    Customer(BankAccount acc, int amt, String name) {
        this.account = acc; this.amount = amt; setName(name);
    }
    public void run() { account.withdraw(amount); }
}
public class SynchronizationDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        Customer customerA = new Customer(account, 700, "Customer-A");
        Customer customerB = new Customer(account, 700, "Customer-B");
        customerA.start(); customerB.start();
    }
}
```

Synchronization – Output & Explanation

Output (With Synchronization):

```
Customer-A trying to withdraw: 700
Customer-A Success! Balance: 300
Customer-B trying to withdraw: 700
Customer-B Insufficient Balance! Balance: 300
```

Concept	Explanation
Monitor Lock	Every Java object has one built-in lock
Acquire Lock	Thread entering `synchronized` method gets the lock
Release Lock	Lock released when method exits (normally or exception)
Race Condition	Multiple threads accessing shared data simultaneously
synchronized keyword	Prevents race conditions by allowing only one thread
Deadlock	Two threads waiting for each other's lock forever

Important Note

Deadlock Warning: Never hold multiple locks in different orders across threads. This causes **deadlock** where both threads wait forever and the program freezes.

Inter-Thread Communication

Definition

Inter-Thread Communication allows synchronized threads to communicate with each other — where one thread **pauses** and another thread is **notified** to continue.

Key Methods (from Object class):

Method	Description
`wait()`	Releases lock and waits until notified
`notify()`	Wakes up ONE waiting thread
`notifyAll()`	Wakes up ALL waiting threads

Syntax:

```
1 synchronized ( obj ) {
2 obj . wait (); // Thread waits
3 obj . notify (); // Notify one
```

⚠ Important Note

Rules:

- Must be called **inside** a `synchronized` method or block
- These are methods of `Object` class, not `Thread` class
- `wait()` **releases** the lock while waiting
- `sleep()` does **NOT** release the lock

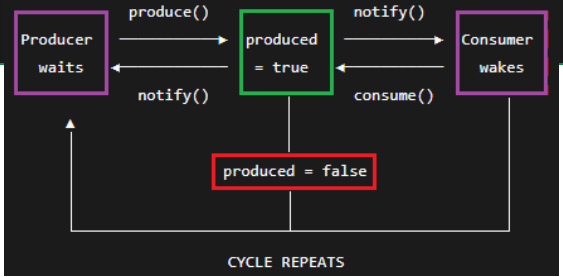
Inter-Thread Communication – Producer-Consumer

```
class SharedResource {
    int data; boolean produced = false;
    public synchronized void produce(int value) {
        while (produced) { try { wait(); } catch (InterruptedException e) {} }
        data = value; produced = true; System.out.println("Produced: " + data);
        notify();
    }
    public synchronized void consume() {
        while (!produced) { try { wait(); } catch (InterruptedException e) {} }
        System.out.println("Consumed: " + data); produced = false;
        notify();
    }
}
class Producer extends Thread {
    SharedResource r;
    Producer(SharedResource r) { this.r = r; }
    public void run() { for (int i = 1; i <= 5; i++) { r.produce(i); } }
}
class Consumer extends Thread {
    SharedResource r;
    Consumer(SharedResource r) { this.r = r; }
    public void run() { for (int i = 1; i <= 5; i++) { r.consume(); } }
}
public class ProducerConsumerDemo {
    public static void main(String[] args) {
        SharedResource r = new SharedResource();
        Producer p = new Producer(r); Consumer c = new Consumer(r);
        p.start(); c.start();
    }
}
```

Inter-Thread Communication – Output & Explanation

Output:

```
Produced: 1     Consumed: 1
Produced: 2     Consumed: 2
Produced: 3     Consumed: 3
Produced: 4     Consumed: 4
Produced: 5     Consumed: 5
```



Key Point

Flow: Producer calls `produce()` → sets `produced=true` → calls `notify()` to wake Consumer → Consumer calls `consume()` → sets `produced=false` → calls `notify()` to wake Producer → cycle repeats.

PART B

Java Database Connectivity (JDBC)

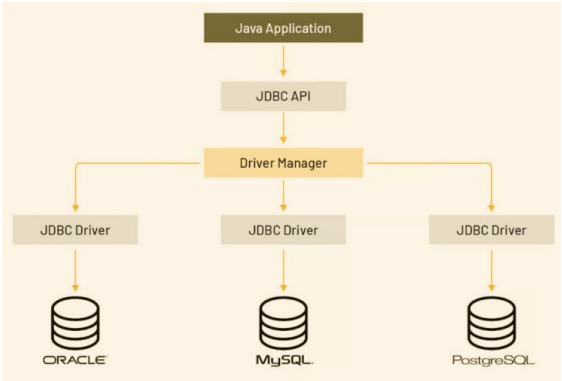
Drivers · Architecture · Classes & Interfaces · Basic Steps · CRUD Operations

Java Database Connectivity (JDBC)

Definition

JDBC (Java Database Connectivity) is a standard Java API that provides a set of **classes and interfaces** to connect Java applications to relational databases (MySQL, Oracle, PostgreSQL) and perform database operations.

What JDBC Does:



JDBC Capabilities:

Operation	SQL Command
Create Table	`CREATE TABLE`
Insert Data	`INSERT INTO`
Read Data	`SELECT`
Update Data	`UPDATE`
Delete Data	`DELETE`

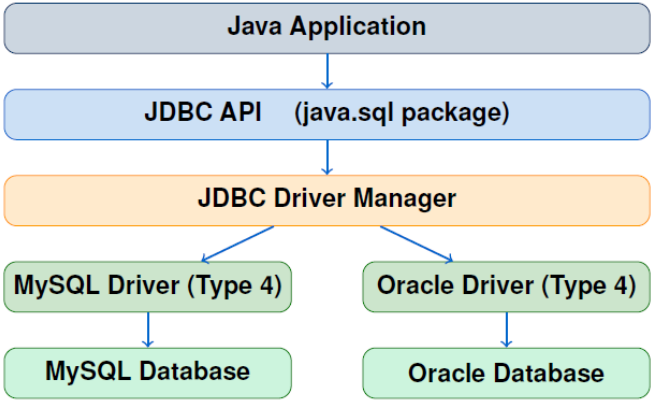
Types of JDBC Drivers

Definition

A **JDBC Driver** is a software component that **translates JDBC calls** into database-specific calls that the database can understand.

Type	Name	How It Works	Advantage	Disadvantage
1	**JDBC-ODBC Bridge**	Converts JDBC to ODBC calls	Works with any ODBC DB	Slow, Windows only
2	**Native-API Driver**	Converts JDBC to native DB calls	Faster than Type 1	Needs native library
3	**Network Protocol**	Sends to middleware server	No client installation	Needs middleware
4	**Thin Driver** ✔	Direct DB protocol conversion	Pure Java, fastest	DB-specific JAR needed

JDBC Architecture



Key Point

Two-Tier: Java App → JDBC Driver → Database (Direct connection)
Three-Tier: Java App → Middleware → JDBC Driver → Database (via Application Server)

JDBC Classes and Interfaces

Important Classes & Interfaces in `java.sql`: Key Methods

Name	Type	Description
`DriverManager`	**Class**	Manages drivers; provides `getConnection()`
`Connection`	**Interface**	Represents a connection to the database
`Statement`	**Interface**	Executes simple SQL queries
`PreparedStatement`	**Interface**	Executes pre-compiled parameterized queries
`CallableStatement`	**Interface**	Executes stored procedures
`ResultSet`	**Interface**	Holds data returned by SELECT queries
`ResultSetMetaData`	**Interface**	Info about ResultSet columns
`DatabaseMetaData`	**Interface**	Info about the database itself
`SQLException`	**Class**	Handles database-related exceptions

"col"

```
Quick Reference:
---------------

1 DriverManager . getConnection(url , user , pass ); // Get connection
2 con . createStatement(); // Create statement
3 stmt . executeQuery(" SELECT ... "); // Returns ResultSet
4 stmt . executeUpdate(" INSERT / UPDATE / DELETE ... "); // Returns int
5 rs. next(); rs. getString(" col"); rs.getInt(" col "); 24 / 36
```

Basic Steps in Developing a JDBC Application

1. Load the JDBC Driver
2. Establish the Database Connection
3. Create a Statement Object
4. Execute a Query
5. Process the Results
6. Close the Connection

```java
// Step 1: Load the Driver
Class.forName("com.mysql.cj.jdbc.Driver");

// Step 2: Establish Connection
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/myDB",
"root", "password");

// Step 3: Create Statement
Statement stmt = con.createStatement();

// Step 4: Execute Query
ResultSet rs = stmt.executeQuery("SELECT * FROM students");

// Step 5: Process Result
while(rs.next()) System.out.println(rs.getString("name"));

// Step 6: Close Resource
con.close();
```

JDBC Connection URL Format

URL Structure:

```
jdbc:mysql://localhost:3306/myDatabase
```

jdbc:	mysql	localhost:3306	myDatabase
JDBC prefix	DB type	Host & Port	DB Name

URL Examples for Different Databases:

Database	JDBC URL	Port
MySQL	jdbc:mysql://localhost:3306/dbname	3306
Oracle	jdbc:oracle:thin:@localhost:1521:xe	1521
PostgreSQL	jdbc:postgresql://localhost:5432/dbname	5432
SQL Server	jdbc:sqlserver://localhost:1433;databaseName=db	1433
SQLite	jdbc:sqlite:database.db	N/A

You must **download and add** the appropriate **JDBC Driver JAR file** to your project classpath before connecting. Example: `mysql-connector-java-8.x.jar`

Creating Database & Table with JDBC – Part 1

```
import java.sql.*;
public class CreateDBTable {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        System.out.println("Driver Loaded ! Connected to MySQL !");

        Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/", "root", "password");
        Statement stmt = con.createStatement();

        stmt.executeUpdate("CREATE DATABASE IF NOT EXISTS StudentDB");
        System.out.println("Database Created !");
        stmt.executeUpdate("USE StudentDB");
        stmt.executeUpdate("CREATE TABLE IF NOT EXISTS students (id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100),
         age INT, marks DOUBLE, city VARCHAR(100))");
        System.out.println("Table Created !");
```

Creating Database & Table with JDBC – Part 2

```
        PreparedStatement ps = con.prepareStatement("INSERT INTO students (name, age, marks, city) VALUES (?,?,?,?)");
        ps.setString(1, "Alice"); ps.setInt(2, 20); ps.setDouble(3, 92.5); ps.setString(4, "Mumbai");
        ps.executeUpdate();
        ps.setString(1, "Bob"); ps.setInt(2, 22); ps.setDouble(3, 87.0); ps.setString(4, "Delhi"); ps.executeUpdate();
        ps.setString(1, "Charlie"); ps.setInt(2, 21); ps.setNull(3, Types.DOUBLE); ps.setString(4, null);
        ps.executeUpdate();
        System.out.println("Records Inserted !\n");

        ResultSet rs = stmt.executeQuery("SELECT * FROM students");
        System.out.println("ID\tNAME\tAGE\tMARKS\tCITY");
        while(rs.next()) {
            System.out.println(rs.getInt("id") + "\t" + rs.getString("name") + "\t" + rs.getInt("age") + "\t" +
            rs.getDouble("marks") + "\t" + rs.getString("city"));
        }

        ps.close(); con.close();
    }
}
```

JDBC Program – Output & Explanation

Output:

```
Driver Loaded!        Connected to MySQL!
Database Created!     Table Created!       Records Inserted!
ID     NAME        AGE    MARKS     CITY
1      Alice       20     92.5      Mumbai
2      Bob         22     87.0      Delhi
3      Charlie     21     95.5      Pune
Connection Closed!
```

Code	Explanation
`Class.forName(DRIVER)`	Dynamically loads JDBC Driver class into JVM
`DriverManager.getConnection`	Creates physical connection to MySQL database
`con.createStatement()`	Creates Statement object to send SQL to DB
`CREATE DATABASE IF NOT EXISTS`	Creates DB only if it doesn't already exist
`AUTO_INCREMENT`	MySQL auto-generates unique ID for each record
`executeUpdate()`	Used for DDL (CREATE) and DML (INSERT/UPDATE/DELETE)
`executeQuery()`	Used for SELECT – returns ResultSet
`rs.next()`	Moves cursor to next row; false when no more rows

PreparedStatement – Parameterized Queries

Definition

PreparedStatement is a pre-compiled SQL statement that accepts **parameters at run-time** using ? placeholders. It is **safer** (prevents SQL Injection) and **faster** for repeated execution.

```
import java.sql.*;
public class PreparedStatementDemo {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/StudentDB", "root", "password");
        PreparedStatement ps = con.prepareStatement("INSERT INTO students (name, age, marks, city) VALUES (?,?,?,?)");
        ps.setString(1, "Diana"); ps.setInt(2, 23); ps.setDouble(3, 88.5); ps.setString(4, "Chennai");
        ps.executeUpdate(); System.out.println("Record 1 Inserted !");
        ps.setString(1, "Bob"); ps.setInt(2, 22); ps.setDouble(3, 75.0); ps.setString(4, "Delhi"); ps.executeUpdate();
        System.out.println("Record 2 Inserted !");
        ps.setString(1, "Evan"); ps.setInt(2, 21); ps.setDouble(3, 82.5); ps.setString(4, "Bangalore");
        ps.executeUpdate(); System.out.println("Record 3 Inserted !");
        ResultSet rs = con.createStatement().executeQuery("SELECT name, age, city FROM students ORDER BY name");
        while (rs.next()) System.out.println(rs.getString("name") + " | Age: " + rs.getInt("age") + " | City: " +
        rs.getString("city"));
        ps.close(); con.close();
    }
}
```

PreparedStatement – Output & Comparison

Output:

```
Record 1 Inserted!
Record 2 Inserted!
Bob     | Age: 22 | City: Delhi
Diana   | Age: 23 | City: Chennai
Evan    | Age: 21 | City: Bangalore
```

Statement vs PreparedStatement:

Feature	Statement	PreparedStatement
SQL Compilation	Compiled every time	Pre-compiled once ✔
Parameters	Hard-coded in SQL	Uses ? placeholders ✔
SQL Injection	Vulnerable	Safe ✔
Performance	Slower (repeated SQL)	Faster for repeated use ✔
Reusability	Low	High ✔
Best For	One-time, simple queries	Dynamic, repeated queries

Summary – Multithreading

Topic	Key Points
Process	Independent program with own memory space
Thread	Lightweight unit of execution within a process
Thread States	New → Runnable → Running → Blocked/Waiting → Terminated
Create Thread	(1) `extends Thread` (2) `implements Runnable`
Priority	MIN=1, NORM=5, MAX=10 | `setPriority()` / `getPriority()`
Synchronization	`synchronized` keyword – prevents race conditions
Monitor Lock	Every object has one lock – only one thread can hold it
Inter-Thread	`wait()` / `notify()` / `notifyAll()`
sleep() vs wait()	`sleep()` holds lock; `wait()` releases lock

Key Point

Quick Rule: Use `implements Runnable` over `extends Thread` for better OOP design. Always use `synchronized` when multiple threads access shared data.

Summary – JDBC

Topic	Key Points
JDBC	Java API for database connectivity (`java.sql.*`)
Driver Types	Type 1: JDBC-ODBC, Type 2: Native, Type 3: Network, Type 4: Pure Java
Best Driver	Type 4 (Pure Java / Thin Driver) – most popular
Architecture	App → JDBC API → Driver Manager → Driver → Database
Key Classes	`DriverManager`, `Connection`, `Statement`, `PreparedStatement`, `ResultSet`
6 Basic Steps	Load Driver → Connect → Create Stmt → Execute → Process → Close
executeQuery	For SELECT – returns `ResultSet`
executeUpdate	For INSERT/UPDATE/DELETE/DDL – returns `int`
PreparedStmt	Parameterized, pre-compiled, safe from SQL Injection

Key Point

Best Practice: Always close JDBC resources in `finally` block (or use *try-with-resources*) to prevent connection leaks.

Important Questions

</> Multithreading:

1. What is the difference between `Thread` and `Runnable`?
2. What is synchronization and why is it needed?
3. What is a deadlock? How can you avoid it?
4. Explain the difference between `sleep()` and `wait()`.
5. What is thread priority? Is it guaranteed?
6. Explain the Thread Life Cycle with all states.
7. What is inter-thread communication? Explain with example.
8. What is a race condition?

JDBC:

1. What are the four types of JDBC drivers?
2. Explain JDBC architecture with a diagram.
3. What is the difference between `Statement` and `PreparedStatement`?
4. What is the difference between `executeQuery()` and `executeUpdate()`?
5. Why should we close JDBC connections?
6. What is SQL Injection? How does `PreparedStatement` prevent it?
7. List the basic steps to develop a JDBC application.
8. Write a program to create a table and insert records.

Quick Reference – Cheat Sheet

Thread Methods:

```
t. start ();
t. sleep (1000) ;
t. join ();
t. setPriority (10) ;
t. getPriority ();
t. getName ();
t. isAlive ();
Thread . currentThread ();
obj . wait ();
obj . notify ();
obj . notifyAll ();
```

JDBC Essentials:

```
Class . forName (" driver ");
Connection con =
DriverManager . getConnection (
url , user , pass );
Statement s =
con . createStatement ();
s. executeQuery (" SELECT ...");
s. executeUpdate (" INSERT ... ");
PreparedStatement ps =
con . prepareStatement ( sql );
ps. setString (1, " value ");
ps. setInt (2, 100) ;
rs. next ();
rs. getString (" col");
rs. close (); s. close ();
con . close ();
```

Thank You!

UNIT IV: Multithreading & JDBC

Key Point

Topics Covered:

- ✔ Process and Thread
- ✔ Thread Life Cycle
- ✔ Creating Threads
- ✔ Thread Priorities
- ✔ Synchronization
- ✔ Inter-Thread Communication
- ✔ JDBC Drivers & Architecture
- ✔ JDBC Classes & Interfaces
- ✔ Basic Steps & CRUD with JDBC

"Code is like humor. When you have to explain it, it's bad." – Cory House